As an example, assume we want to prove $\vdash (3 + 2) = 5$ using the definition of 4 and 5 as respective successors of 3 and 4. As a first step, we should use an equality transitivity theorem such as:

```
[[
  |- A = B            # first hypothesis
  |- C = B            # second hypothesis
]]
|- A = C              # conclusion
```

To apply the transitivity theorem, we need to provide a substitutions that substitutes A with $(3 + 2)$ and B with 5 such that the conclusion of the theorem unifies to the current goal. We are left with providing a substitution for B which can hardly be discovered mechanically (hence the appeal to use generative language modeling). We can substitute B with $(4 + 1)$ as is the case in the actual proof[2] in Metamath's *set.mm* library.

Putting it all together, the goal here is:

```
|- ( 3 + 2 ) = 5
```

The proof step we apply:

```
[[
  |- A = B            # first hypothesis
  |- C = B            # second hypothesis
]]
|- A = C              # conclusion
{{ A : ( 3 + 2 ) }}   # substitution of A
{{ B : ( 4 + 1 ) }}   # substitution of B
{{ C : 5 }}           # substitution of C
```

And finally the new subgoals are:

```
|- ( 3 + 2 ) = ( 4 + 1 )
|- ( 4 + 1 ) = 5
```

Applying the following proof step with no hypothesis (the definition of 5[3]) to the second subgoal allows us to prove it.

```
[[ ]] |- ( 4 + 1 ) = 5
```

Note that this proof step has no hypothesis and no substitution involved. It therefore closes that branch of the proof tree. From there the proof can be continued with the first subgoal, proving backward, until no subgoal is left. Also note that a proof for a given theorem of the library can only use theorems proven before the appearance of the theorem to prove; we enforce that constraint when benchmarking our models despite them being trained on the library as a whole.

In most formal systems, a *proof step*, consists of a *goal* and a mechanism that, given a goal produces new subgoals, generally referred to as a *tactic*. In Metamath, there is only one type of *tactic* based on substitution as illustrated above. Additionally since the substituted theorem must unify to the current goal, the current *goal* can be deduced from the *tactic* itself (theorem and substitution pair), which is not generally the case in other systems. As such, we'll use *tactic* and *proof step* interchangeably in the rest of the paper.

This informal presentation of Metamath is sufficient to understand the objectives we use to train our models. A more formal definition of Metamath's meta-logic can be found in the Metamath Book [42].

3.2 Dataset

Metamath's *set.mm* uses a binary compressed format to represent proofs of statements. We process the library and extract a dataset of proof steps, stored as JSON blobs using the representation presented

[2] *Metamath Proof Explorer - 3p2e5* `http://us.metamath.org/mpeuni/3p2e5.html`

[3] *Metamath Proof Explorer - df-5* `http://us.metamath.org/mpeuni/df-5.html`

above. For each proof step we store a *GOAL*, a *PROOFSTEP* and a reference to the parent goal if any, encoding the tree structure of the proofs:

```
{
  "proof_label": "unidmrn",
  "goal": "[[ ]] |- U. U. ‘’ A = ( dom A u. ran A )",
  "proof_step": "[[ |- A = B |- C = B ]] |- A = C \\
     {{ A : U. U. ‘’ A }} \\
     {{ B : ( ran ‘’ A u. dom ‘’ A ) }} \\
     {{ C : ( dom A u. ran A ) }}",
  "proof_step_hash": "37yZVNorgF8=",
  "parent_hash": ["n4Kl7judEN4="]
}
```

The dataset contains $\sim 3m$ of such proof steps for $\sim 38k$ theorems (different proof labels). We split that dataset between a *train* set and two *valid* and *test* sets each containing $\sim 1k$ proofs sampled randomly ($\sim 90k$ proof steps each).

3.3 Glossary

term	A string that comply to the Metamath grammar.
statement or *proposition*	A potentially empty set of hypotheses (terms) and a conclusion (term) entailed by the hypotheses.
theorem	A proven statement.
axiom	An assumed statement.
goal	A statement in the context of a proof search.
substitutions	A list of pairs of variables and terms (to substitute the variables within a theorem or an axiom).
tactic	A theorem and substitutions that unify to a goal.
subgoals	Goals generated by a tactic (the substituted hypotheses of the tactic's theorem).
proof step	A goal and a tactic, potentially generating new subgoals.
proof	A tree of goals and tactics whose root is the demonstrated theorem; leaves of the tree are tactics with no subgoals or goals that are hypotheses of the root theorem.

4 Model

4.1 Architecture

We use decoder-only Transformers similar to GPT-2 [20] and GPT-3 [21]. The largest model we study has 36 layers and 774m trainable parameters.

4.2 Training Objective

The *proofstep objective* we use for training is a conditional language modeling objective that is asked to generate the *PROOFSTEP* given a *GOAL*, which is directly applicable to proof searches. To do so, we format our data in the following way:

```
GOAL <GOAL> PROOFSTEP <PROOFSTEP><EOT>
```

There is one such objective for each JSON line in our dataset. We train with only one sentence per context (no-chunking), masking the rest of the context by assigning a loss weight $w_{loss} = 0$. As we train we track the valid loss and sequence accuracy while masking the query part of the objective:

```
GOAL <GOAL> PROOFSTEP
```

We regularize the training by early-stopping at the point of minimum valid loss and applying a weight decay $wd = 0.1$.

Here is a randomly sampled context as presented to our models for training:

```
GOAL [[ ]] |- ( ( ( J e. Nrm /\ f e. ( J Homeo K ) ) /\ ( x e. K /\  y e. (
( Clsd ‘ K ) i^i ~P x ) ) ) -> ( ‘’ f " x ) e. J ) PROOFSTEP [[ |- ( ph ->
ps ) |- ( ph -> ch ) |- ( ( ps /\ ch ) -> th ) ]]  |- ( ph -> th ) {{ ch :
x e. K }} {{ ph : ( ( J e. Nrm /\ f e.  ( J Homeo K ) ) /\ ( x e. K /\ y e.
( ( Clsd ‘ K ) i^i ~P x ) ) ) }}  {{ ps : f e. ( J Cn K ) }} {{ th : ( ‘’ f
" x ) e. J }} <|endoftext|>
```

4.3 Proof Search

4.3.1 Goal Expansion

We find proofs by running proof searches. A proof search maintains a proof tree and a queue of open goals sorted by their cumulative logprob, initialized with the root goal that we wish to demonstrate (see figure 1). The cumulative logprob of a goal is defined by the sum of the logprobs of the tactics that were used to reach that goal from the root goal. Intuitively we expand goals for which the generative model is the most confident globally. This has a tendency to explore breadth first as deeper goals have more parent tactics and therefore typically a higher cumulative logprob.

Each time we expand an open goal we sample $e = 32$ tactics (the *proofstep objective* described above) from the model at temperature $t = 1.0$, deduplicate them, and apply the valid tactics (of which there are at most e) to the goal being expanded. Each successful tactic application generates new subgoals that are added to the proof tree and the proof search queue. The expanded goal is then removed from the queue. Note that the subgoals associated with a successfully applied tactic all share the same cumulative logprob and will eventually be expanded together (as subgoals generated from their own expansion will mechanically have a higher cumulative logprob, and will therefore be inserted behind in the queue). We denote the process of selecting the minimal cumulative logprob goal and expanding it as a proof search *expansion*.

Each proof search involves $d = 128$ goal expansions, so proofs generated have at most d proof steps. When evaluating our models, we attempt a proof search for each statement in the *valid* set $a = 4$ times, starting from an empty proof tree each time. In the above, a, e, and d are hyperparameters of the search process that we can vary to achieve better performance (at the cost of more compute), but keep constant as we compare models.

4.3.2 Formal Verifier

Performing such proof searches requires to tightly couple a Metamath verifier with our models. We implemented a Metamath kernel in Python to avoid the performance cost and brittleness of interacting with an external kernel over its REPL through standard I/O. It also provides us with a flexible environment to experiment with new ideas in ways that were not anticipated by existing verifiers. The kernel consists of a modified *LR(0)* parser to check that terms generated by our models comply with the Metamath grammar, along with *Goal* and *Tactic* objects that implement the Metamath substitution and represent proof trees. Our implementation is capable of exporting our in-memory representations to both our JSON marshalled format and the official *set.mm* proof format. The latter allows us to verify the proofs we generate with an external Metamath kernel implementation such as `mmverify.py` or `metamath-exe`.

Collectively, this proof search procedure and the formal verifier tied with it are what we refer to as the *GPT-f* automated prover.

4.4 Evaluation

We report the performance $\text{Perf}^{valid}_{a,e,d}(\theta)$ of a model θ, as the percentage of proofs found by this procedure within the *valid* or *test* set. We evaluate our models on the *valid* set of $\sim 1k$ theorems and once at the end of this paper on the held-out *test* set.